

Absolutely. I'd make this as a **daily one-page interview dashboard** — something you can read every morning in 2–3 minutes before starting preparation.

Full Stack Developer — Daily Interview One-Pager

My Goal

Become interview-ready as a Full Stack Developer with strong Backend expertise and confident React knowledge.

Rule: I don't need to know everything. I need to **understand, recall, code, and explain** what I know.

My Daily 2-Hour Routine

Time	Focus	Daily Action
45 min	 Backend	Python / FastAPI / SQL / APIs
30 min	 React	Learn → Code → Explain
30 min	 DSA	1–2 problems
20 min	 System Design	One concept
15 min	 Interview	Speak answers aloud

Golden Rule

30% Learn → 70% Practice + Recall + Explain

Don't just watch tutorials.

45 MIN — BACKEND

Rotate through:

Python

- Data Types
- Functions
- OOP
- `*args` / `**kwargs`
- Decorators
- Iterators / Generators
- Context Managers
- Exception Handling
- Async / Await

- GIL
- Threading vs Multiprocessing

FastAPI

- Routing
- Pydantic
- Dependency Injection
- Middleware
- Authentication
- JWT
- Async endpoints
- Background Tasks
- Error Handling
- Pagination
- Service / Repository Architecture

Database

- SQL
- Joins
- Indexes
- Transactions
- Isolation Levels
- Query Optimization
- SQLAlchemy
- MongoDB
- Aggregation
- Redis

Ask myself:

"Can I explain this to an interviewer without looking at notes?"

If YES →  If PARTIAL →  If NO → 



30 MIN — REACT

Monday

React Core

Components → Props → State → Events → Forms → Lists

Tuesday

Hooks

`useState` → `useEffect` → `useRef` → `useMemo` → `useCallback`

Wednesday

Advanced React

Context → Custom Hooks → Lifecycle → Re-rendering → State Management

Thursday

React + Backend

React → API → FastAPI → Database → Response → UI

Friday

Performance

Memoization → Lazy Loading → Code Splitting → Debouncing → Virtualization

Saturday

Build

Build one small feature without following a tutorial.

Sunday

Interview

Answer React questions from memory.

React Questions I Must Be Able To Answer

1. What causes a React component to re-render?
 2. `useState` vs `useRef`?
 3. `useEffect` and dependency array?
 4. `useMemo` vs `useCallback`?
 5. What is `React.memo`?
 6. Why are keys required in lists?
 7. Controlled vs uncontrolled components?
 8. What is Context API?
 9. What are custom hooks?
 10. What is code splitting?
 11. What is lazy loading?
 12. CSR vs SSR?
 13. How do you optimize a slow React application?
 14. How does React communicate with a backend API?
-



30 MIN — DSA

Don't randomly solve problems.

Follow patterns:

```
Arrays
↓
Strings
↓
HashMap
↓
Two Pointers
↓
Sliding Window
↓
Stack / Queue
↓
Binary Search
↓
Linked List
↓
Trees
↓
Graphs
↓
Dynamic Programming
```

Daily target

1 easy + 1 medium

But understanding the pattern is more important than the number of problems.

Ask:

What pattern does this problem use?



20 MIN — SYSTEM DESIGN

One topic per day.

Core

- API Design
- Load Balancer
- Caching
- Redis

- Database Indexing
- SQL vs NoSQL
- Message Queue
- Kafka
- RabbitMQ
- Rate Limiting
- Authentication
- Microservices
- Logging
- Monitoring

Then design:

```
Client
  ↓
Load Balancer
  ↓
FastAPI
  ↓
Redis
  ↓
MySQL
  ↓
Queue
  ↓
Worker
```

Ask:

What happens if traffic becomes 10x?

What happens if the database goes down?

Where can I cache?

Where can I use a queue?

How do I scale this service?



15 MIN — INTERVIEW MODE

Close all notes.

Pick 3 questions.

Answer each **out loud**.

Answer structure

1. Definition
2. Why it is needed
3. How it works
4. Real-world example
5. Trade-offs

Example:

What is Redis?

Don't just say:

"Redis is an in-memory database."

Instead:

"Redis is an in-memory key-value store commonly used for caching, sessions, rate limiting and temporary data. Because data is primarily served from memory, it provides very low latency. In a FastAPI application, I could use Redis to cache frequently requested company data and reduce database load. The trade-off is that cached data can become stale and Redis requires additional infrastructure."

That's an interview answer.



My Daily Rules

Rule 1

Don't study everything every day.

Focus on one topic.

Rule 2

Don't just watch. Code.

Rule 3

Don't just code. Explain.

Rule 4

Don't memorize interview answers.

Understand the concept.

Rule 5

React doesn't need 3 hours/day.

30 minutes consistently is enough to keep your frontend skills alive.

Rule 6

Backend is my strength.

Go deeper there.

Rule 7

Every week, build something.

Real coding > tutorial watching.

Rule 8

Every Sunday, test myself.

If I can't explain it without notes, I don't know it well enough yet.



Daily Score

At the end of the day:

Backend	★★★★★
React	★★★★★
DSA	★★★★★
System Design	★★★★★
Interview	★★★★★

And write only **one sentence**:

Today I learned _____.

Tomorrow, start by explaining yesterday's topic **without looking at your notes**.



Remember

```
DON'T AIM FOR
  ↓
"I know everything"
  ↓
  AIM FOR
  ↓
```

```
"I can understand it,  
code it and explain it."
```

Your target profile

Strong Python Backend + Solid React + Good DSA + Strong System Design + Clear Communication

That's a very strong **Full Stack Developer** interview profile.